

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0510

Slot A

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Case Study

Code of Conduct:

I P. Akhila / 192472320 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Akhila

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail.

Analyze the problem and explain how preprocessing enhances edge detection results.

CASE STUDY 1:

Edge Detection in Low Contrast Images

Introduction

Edge detection is an important computer vision technique used to identify the boundaries of objects in an image. In a low-contrast image, the difference between neighbouring regions is very small, so edge detection algorithms may fail to identify important boundaries clearly.

Problem Statement

The given image has low contrast, causing the edge detection algorithm to miss weak or unclear edges. The main challenge is to improve the image contrast before applying edge detection so that important boundaries can be detected properly.

Case Analysis

Low contrast results in only small intensity changes between an object and its background. Since edge detectors depend on intensity changes, weak boundaries may appear broken or may not be detected at all.

For example, in a medical image, two nearby tissues may have similar intensity values. Directly applying an edge detector can therefore produce incomplete boundaries.

Computer Vision Concepts Applied

- **Contrast enhancement** increases the difference between neighbouring intensity regions.
- **CLAHE** improves contrast locally in different parts of the image.
- **Gaussian filtering** can reduce unwanted noise before edge detection.
- **Canny edge detection** can then identify the enhanced object boundaries.

Proposed Solution

The following processing sequence can be used:

Input Image → CLAHE → Gaussian Filtering → Canny Edge Detection → Final Edge Image

CLAHE is suitable because it improves **local contrast**, which is useful when different regions of the image have different brightness or contrast levels.

Expected Outcome

After preprocessing, weak boundaries should become more visible. Canny edge detection is expected to produce clearer, stronger, and more continuous edges, with fewer missed boundaries.

Conclusion

The low contrast of the image is the main reason for poor edge detection. Applying contrast enhancement and noise reduction before Canny edge detection can improve the visibility of weak boundaries and make the detected edges more reliable.

2. Corner Detection in Textured Regions

A system applies corner detection in highly textured regions, resulting in excessive keypoints. Analyze the issue and suggest methods to improve feature selection.

CASE STUDY 2:

Corner Detection in Textured Regions

Introduction

Corner detection is used to identify important points in an image that can be used for feature extraction, matching, and object recognition. In highly textured regions, many intensity changes are present close to each other, so the corner detector may identify an excessive number of key points.

Problem Statement

The system is applying corner detection in a highly textured region and is producing too many key points. Many of these key points may be repetitive or less useful, which can increase processing time and make later feature matching less reliable.

Case Analysis

Textured areas such as brick walls, grass, fabric, or repeated patterns contain many small changes in intensity. A corner detector can treat these changes as separate corners even though they may not provide useful information for identifying an object.

If all detected points are retained, the system has to process a large number of features. This increases computational cost and may also introduce redundant or unreliable points during matching.

Computer Vision Concepts Applied

- Corner detection identifies points where image intensity changes significantly in more than one direction.
- Corner response indicates how strong or reliable a detected corner is.
- Thresholding can remove weak keypoints.
- Non-maximum suppression helps eliminate nearby redundant points.
- Feature selection retains only the most distinctive and useful key points.

Proposed Solution

The detected key points should be filtered before they are used for further processing:

Input Image → Corner Detection → Remove Weak Key points → Remove Redundant Points → Select Strong Key points → Feature Matching

A suitable response threshold can be used to remove weak corners, while a minimum distance between selected points can prevent too many keypoints from being concentrated in the same textured region.

Expected Outcome

The number of key points should be reduced without losing the important corners of the image. The selected features should be stronger, better distributed, and more useful for subsequent feature matching or object recognition.

Conclusion

The problem is not the failure of corner detection but the excessive number of detected features. Filtering weak and redundant key points provides a smaller and more meaningful feature set, improving computational efficiency and the reliability of further image analysis.

3. Feature Matching with Noise

Feature matching between two images fails due to noise in one image.

Analyze the impact of noise and explain how preprocessing improves matching accuracy.

CASE STUDY 3:

Introduction

Feature matching is used to find corresponding points between two images. When one of the images contains noise, the appearance of local regions changes, which can affect the extracted features and make it difficult to find correct matches.

Problem Statement

The feature matching process fails because one of the input images contains noise. The noise changes pixel intensities and affects the local features, resulting in incorrect matches or failure to identify corresponding points.

Case Analysis

Noise introduces unwanted intensity variations that are not part of the actual object or scene. Since feature descriptors are calculated from local image regions, these variations can change the descriptor even when two regions represent the same object.

For example, if two images of the same object are captured and one image has sensor noise, corresponding regions may produce different feature descriptions. This can lead to fewer correct matches and more false matches.

Computer Vision Concepts Applied

- Noise removal reduces unwanted intensity variations before feature extraction.
- Feature detection identifies important points that can be compared between images.
- Feature descriptors represent the local appearance around detected keypoints.
- Feature matching compares descriptors to find corresponding points.
- RANSAC can be used to remove incorrect matches after matching.

Proposed Solution

The noisy image should first be preprocessed before feature extraction:

Two Input Images → Noise Removal → Feature Detection → Descriptor Extraction → Feature Matching → Remove False Matches → Final Matches

A suitable denoising filter can reduce noise while preserving important image structures. After preprocessing, feature descriptors can be extracted from both images and matched. RANSAC can then be used to reject geometrically inconsistent matches.

Expected Outcome

Preprocessing should make corresponding regions more similar and improve descriptor reliability. Therefore, the system should obtain more correct matches and fewer false matches compared with matching directly on the noisy image.

Conclusion

Noise can significantly affect feature descriptors and reduce matching accuracy. Removing unwanted noise before feature extraction, followed by suitable matching and outlier rejection, provides more reliable feature correspondences between the two images.

4. PCA and Information Loss

A system applies PCA aggressively and loses important feature details.

Analyze the trade-off between dimensionality reduction and information loss, and justify optimal component selection.

CASE STUDY 4:

Introduction

Principal Component Analysis (PCA) is a dimensionality reduction technique used to represent high-dimensional image features using fewer components. It can reduce computational complexity, but applying PCA too aggressively may remove important information required for accurate image analysis.

Problem Statement

The system applies PCA aggressively and loses important feature details. Although the dimensionality of the data is reduced, some useful information is discarded, which can affect the performance of later tasks such as classification, recognition, or matching.

Case Analysis

PCA arranges the transformed components according to the amount of variance they represent. The first few components generally contain the largest amount of information, but the remaining components may still contain useful details.

If too few components are selected, important variations in the image features may be lost. On the other hand, keeping too many components reduces the benefit of dimensionality reduction and increases computational requirements.

Therefore, the main challenge is to find a balance between reducing feature dimensions and preserving important information.

Computer Vision Concepts Applied

- PCA transforms high-dimensional features into a smaller set of principal components.
- Dimensionality reduction reduces storage and computational requirements.
- Explained variance indicates how much information is retained by each component.
- Cumulative explained variance helps determine how many components should be kept.
- Performance evaluation can verify whether the reduced features are still sufficient for the required task.

Proposed Solution

The number of PCA components should be selected based on the amount of information that needs to be preserved rather than choosing a very small number directly.

Original Features → Standardization → PCA → Calculate Explained Variance → Select Suitable Components → Evaluate Performance

For example, components can initially be selected to retain around 90–95% of the explained variance, and the result can then be checked using the actual application performance.

Expected Outcome

The selected PCA representation should reduce the number of features while retaining most of the important information. The system should require less computation without causing a significant decrease in recognition, classification, or matching performance.

Conclusion

PCA provides a useful way to reduce feature dimensions, but excessive reduction can cause information loss. Selecting components using explained variance and validating the resulting performance provides a better balance between efficiency and information preservation.

5. Combined Feature Detection and Matching

A system detects features correctly but fails during matching due to incorrect descriptor selection.

Analyze the issue and explain how appropriate feature descriptors improve matching performance.

CASE STUDY 5:

Introduction

Feature detection and feature matching are important stages in computer vision. Feature detection identifies important points in an image, while a feature descriptor describes the local region around each point so that corresponding features can be found between two images.

Problem Statement

The system is able to detect features correctly, but the matching stage fails because an inappropriate feature descriptor has been selected. Therefore, the problem is not with detecting the keypoints, but with representing and comparing them effectively.

Case Analysis

A detector only identifies the location of important points; it does not determine how those points should be compared with points in another image. The descriptor must capture enough information about the local region to distinguish one feature from another.

If an unsuitable descriptor or matching method is used, corresponding points may receive different descriptor values. This can result in fewer correct matches and a large number of false matches.

Computer Vision Concepts Applied

- Feature detection identifies distinctive points such as corners and textured regions.
- Feature descriptors represent the visual information around each detected point.
- Feature matching compares descriptors to find corresponding points in two images.
- SIFT is suitable when robustness to scale and rotation changes is important.
- ORB provides faster feature description and is useful when computational efficiency is important.

Proposed Solution

The descriptor should be selected according to the image conditions and the feature detector:

Input Images → Feature Detection → Select Suitable Descriptor → Descriptor Extraction → Appropriate Matcher → Remove False Matches → Reliable Matches

For example, SIFT can be used when accuracy and robustness are more important, while ORB can be considered when faster processing is required. The matching method should also be compatible with the selected descriptor.

Expected Outcome

Using a suitable descriptor should increase the number of correct correspondences and reduce false matches. The final output should show reliable matching lines between corresponding features in the two images.

Conclusion

Correct feature detection alone is not sufficient for successful image matching. Selecting an appropriate descriptor and a compatible matching method improves the reliability and accuracy of feature matching, especially when the images have changes in scale, rotation, or appearance.